

**Московский Физико-Технический Институт
Физтех-школа Аэрокосмических технологий
Институт Аэромеханики и Летательной Техники**



ФАКТ

**Архитектура Компьютера
и Операционные Системы.**

Часть 2. Основы операционных систем

**Лекция 13: Введение в сетевое
программирование в UNIX.
Часть 4/4: Unix domain sockets**

**Новиков Андрей Валерьевич
д.ф.-м.н.**

Жуковский

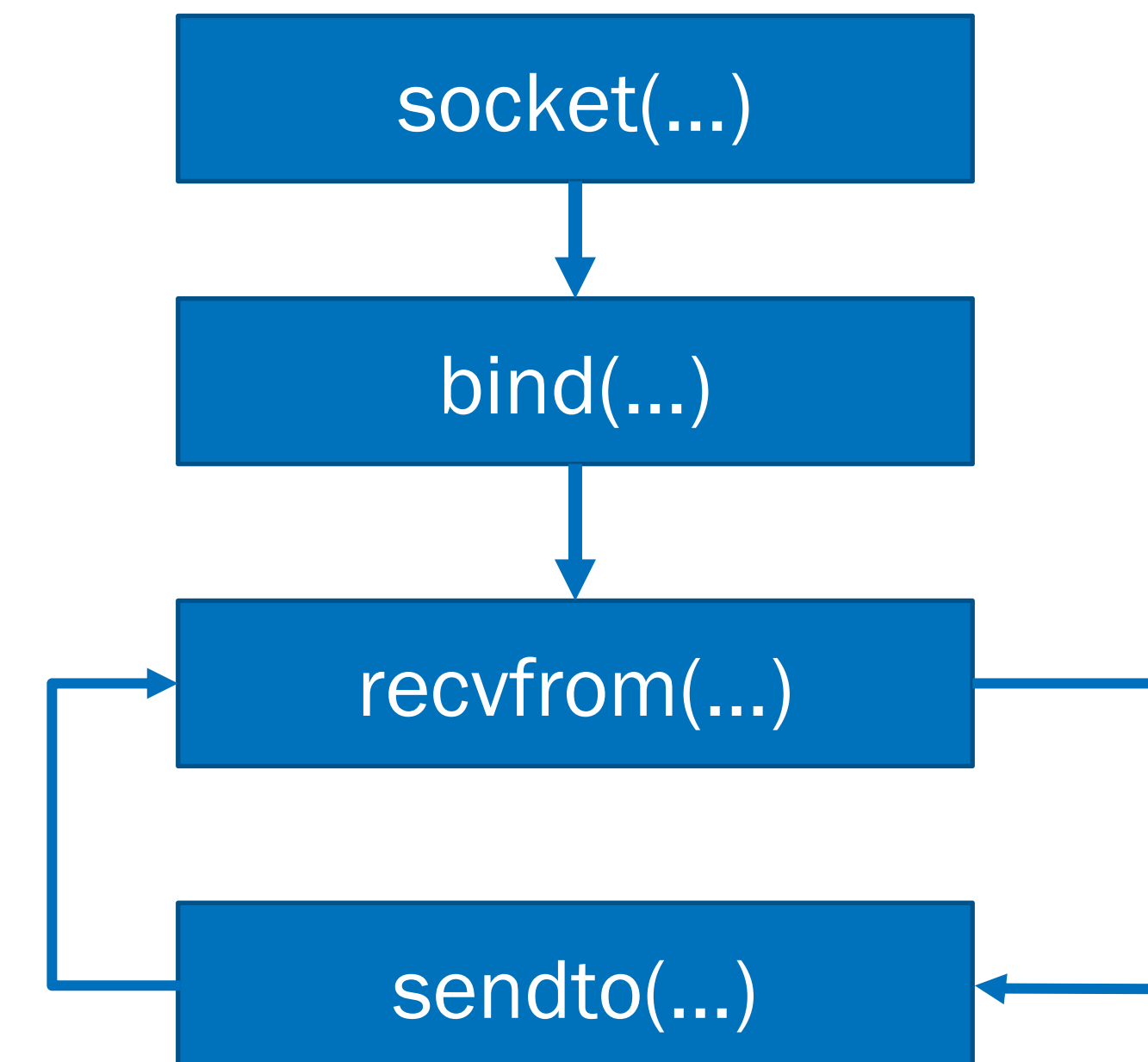
Связь между удаленными процессами с помощью UNIX domain протоколов

Сетевые протоколы UNIX domain

- ❑ Общение процессов в пределах одной операционной системы, локального компьютера в рамках сетевого интерфейса.
- ❑ Реальное сетевое устройство не задействуется, вся передача данных проходит через адресное пространство ядра подобно **pipe**. Малые накладные расходы, большая скорость.
- ❑ Имеет 2 протокола: потоковый и датаграммный (эквивалент TCP и UDP).
- ❑ Позволяет использовать универсальный подход в клиент-серверных программах, которые могут работать как локально, так и удалённо, например X-server
- ❑ Адресация:
 - полный адрес = ~~удалённый адрес (IP)~~ + локальный адрес (порт)
 - полный адрес = локальный адрес = имена спец. файлов
 - при привязывании сокета к адресу создаётся специальный файл-метка типа socket

Отличие интерфейса от ТСП/IP

- ❑ Другой тип сокета – другие аргументы у `socket()`
- ❑ Другое пространство адресов – `struct sockaddr` содержит другие данные, другие поля.
 - меняется способ вычисления длины адреса
 - меняются аргументы у `bind()`, `recvfrom()`, `sendto()`, `connect()`.
- ❑ Другие заголовочные файлы — вместо ``netinet/in.h`` и ``arpa/inet.h`` файл ``sys/un.h``
- ❑ Всё!



Создание сокета

```
#include <sys/types.h>
#include <sys/socket.h>

int socket(int domain,
           int type, int protocol);
```

Создание виртуального коммуникационного узла в ОС.

- ❑ **domain** – семейство протоколов для передачи:
PF_UNIX – семейство протоколов UNIX domain.
- ❑ **type** – семантика обмена информацией
SOCK_STREAM – связь с помощью установления виртуального соединения;
SOCK_DGRAM – обмен через сообщения.
- ❑ **protocol** – конкретный протокол для выбранного семейства и способа обмена, если их несколько
= 0 для UNIX domain
- ❑ Возвращаемое значение
fd >= 0 – файловый дескриптор;
< 0 – ошибка, расшифровка в **errno**

Привязка сокета к адресу

```
#include <sys/types.h>
#include <sys/socket.h>

int bind(int sockd,
        struct sockaddr* my_addr,
        int addrlen);
```

Привязка сокета к определенному полному адресу сети.

- ❑ **sockd** - дескриптор коммуникационного узла (сокета)
- ❑ **my_addr** – указатель на структуру с информацией о том, куда именно мы хотим привязать сокет – адрес сокета. Структура должна быть конкретизирована в зависимости от используемого семейства протоколов.
- ❑ **addrlen** - фактическая длина структуры с адресом, каждый раз разная для UNIX Domain.
- ❑ Возвращаемое значение
 - 0 – успех,
 - 1 – ошибка, расшифровка в **errno**

Адрес для семейства **UNIX domain**

```
#include <sys/un.h>

struct sockaddr_un {
    short sun_family;
    char sun_path[108];
};

SUN_LEN(struct sockaddr_un*);
```

Адрес сокета для семейства UNIX_DOMAIN

- ❑ **sun_family** – выбранное семейство, всегда **AF_UNIX**
- ❑ **sun_path** – путь к специальному файлу типа socket
- ❑ **SUN_LEN()** – макрос для вычисления фактической длины адреса:
= sizeof(sun_family) + strlen(sun_path).

Отправка датаграмм

```
#include <sys/types.h>
#include <sys/socket.h>

int sendto(int sockd,
           char *buff, int nbytes,
           int flags,
           struct sockaddr* to,
           int addrlen);
```

Отправка *датаграмм* по протоколу UDP. Может блокироваться при отсутствии места в сетевом буфере.

- ❑ **sockd** — дескриптор сокета, через который отсылается информация.
- ❑ **buff** — адрес области памяти с информацией для передачи
- ❑ **nbytes** — количество байт, которое должно быть передано
- ❑ **to** — *адрес сокета* получателя
- ❑ **addrlen** — фактическая длина структуры с адресом
- ❑ **flags** — режим, 0 – по-умолчанию.
- ❑ Возвращаемое значение:
>= 0 количество переданных байт, < 0 - ошибка

Приём датаграмм

```
#include <sys/types.h>
#include <sys/socket.h>

int recvfrom(int sockd,
             char *buff, int nbytes,
             int flags,
             struct sockaddr *from,
             int *addrlen);
```

Приём *датаграмм* по протоколу UDP. Блокируется при отсутствии пришедших датаграмм в сокете.

- ❑ **sockd** — дескриптор сокета, через который принимается информация.
- ❑ **buff** — адрес области памяти, куда помещается информация
- ❑ **nbytes** — макс. количество байт в буфере
- ❑ **from** — *адрес сокета* отправителя, может быть NULL
- ❑ **addrlen** — длина структуры с адресом. До вызова – максимальная длина, после вызова – фактическая длина
- ❑ **flags** — режим, 0 – по-умолчанию.
- ❑ Возвращаемое значение:
>= 0 количество принятых байт, < 0 - ошибка

Соединение клиента с сервером

```
#include <sys/types.h>
#include <sys/socket.h>

int connect(int sockd,
            struct sockaddr *servaddr,
            int addrlen);
```

Организация связи клиента с сервером. Обычно устанавливают логическое соединения, хотя можно применять и при связи на датаграммах.

Блокируется до установления соединения или до окончания таймаута.

- ❑ **sockd** — дескриптор сокета
- ❑ **servaddr** — адрес сокета сервера
- ❑ **addrlen** — фактическая длина структуры с адресом
- ❑ Возвращаемое значение:
0 - успех, < 0 - ошибка

Ожидание соединения на сервере

```
#include <sys/types.h>
#include <sys/socket.h>

int listen(int sockd,
           int backlog);
```

Перевод TCP-сокета в пассивное (слушающее) состояние и создание очереди для порождаемых при установлении соединения сокетов, находящихся в состоянии не полностью установленного соединения и полностью установленного соединения.

- ❑ **sockd** — дескриптор сокета
- ❑ **backlog** — максимальный размер очередей для сокетов, находящихся в состояниях полностью и/или не полностью установленных соединений (в зависимости от версии Unix).
- ❑ Возвращаемое значение:
0 - успех, < 0 - ошибка

Приём соединения на сервере

```
#include <sys/types.h>
#include <sys/socket.h>

int accept(int sockd,
           struct sockaddr *cliaddr,
           int *clilen);
```

Приём полностью установленного соединения. Возвращает дескриптор для первого присоединенного сокета из очереди соединений, одновременно удаляя его из очереди. Или ожидает появления соединения.

- ❑ **sockd** — дескриптор сокета
- ❑ **cliaddr** — адрес сокета отправителя, может быть NULL
- ❑ **addrlen** — длина структуры с адресом. До вызова – максимальная длина, после вызова – фактическая длина
- ❑ Возвращаемое значение:
 - ≥ 0 – дескриптор сокета для обмена данными,
 - < 0 - ошибка

Литература

1. В.Е. Карпов, К.А. Коньков - Основы операционных систем // М.: Национальный открытый университет «ИНТУИТ», 2016
<https://intuit.ru/studies/courses/2192/31/info>
2. В.Е. Карпов, К.А. Коньков - Основы операционных систем. Практикум // М.: Национальный открытый университет «ИНТУИТ», 2016
<https://intuit.ru/studies/courses/2249/52/info>
3. Google 😊

